

Example showing how C-code is converted to MIPS assembly language

Section 6.6 of the Harris and Harris Book gives the following example of C code:

```
/*
 * Example C code as given in Section 6.6 of Harris & Harris Book.
 */

int f, g, y; // global variables

int main(void)
{
    f = 2;
    g = 3;
    y = sum(f, g);

    return y;
}

int sum(int a, int b) {
    return (a + b);
}
```

On a MIPS-based Ci40 single-board computer running Linux we can turn this into MIPS assembly code using “`gcc -S test.c`” to obtain a “`test.s`” assembly file containing MIPS assembly.

I’ve highlighted and annotated some of the sections in this resulting “`test.s`” file below:

```
.file 1 "test.c"
.section .mdebug.abi32
.previous
.nan legacy
.module    fp=32
.module    nooddspreg
.abicalls
```

```

.comm f,4,4      # These are like ".word" assembly directives in
                  # that they allocate data memory to store the
.comm g,4,4      # global variables f, g and y.

.comm y,4,4
.text            # Assemble into the ".text" segment.
.align          2
.globl          main    # Main is a global symbol called by the OS.
.set nomips16
.set nomicromips
.ent main
.type main, @function
main:            # Main label where program execution starts.
.frame          $fp,32,$31    # vars= 0, regs= 2/0, args= 16, gp= 8
.mask 0xc0000000,-4
.fmask         0x00000000,0
.set noreorder
.set nomacro
addiu $sp,$sp,-32 # Moves stack pointer down 32 bytes to
                  # make space on the stack for use by main.
sw          $31,28($sp) # Stores return address onto the stack.
sw          $fp,24($sp) # Stores frame pointer onto the stack.
move $fp,$sp
lui $28,%hi(__gnu_local_gp)
addiu $28,$28,%lo(__gnu_local_gp)
.cprestore 16
lw          $2,%got(f)($28)    # These lines set "f = 2".
li          $3,2              # 0x2
sw          $3,0($2)
lw          $2,%got(g)($28)    # Similarly sets "g = 3".
li          $3,3              # 0x3
sw          $3,0($2)
lw          $2,%got(f)($28)
nop
lw          $3,0($2)
lw          $2,%got(g)($28)
nop
lw          $2,0($2)
nop
move $5,$2          # Sets arguments $a0 (register 5) and
move $4,$3          # $a1 (register 4) before ...
.option        pic0
jal sum            # ... calling the sum function.
nop

.option        pic2
lw          $28,16($fp)
move $3,$2          # This moves the result of the function ($v0)

```

```

lw    $2,%got(y)($28) # into a register which is stored as "y".
nop
sw    $3,0($2)
lw    $2,%got(y)($28)
nop
lw    $2,0($2)
move  $sp,$fp
lw    $31,28($sp) # Restore the return address register $ra.
lw    $fp,24($sp) # Restore the frame pointer.
addiu $sp,$sp,32  # Restore the stack pointer by adding 32.
j     $31         # Before returning from main.
nop

.set macro
.set reorder
.end main
.size main, .-main
.align    2
.globl    sum      # Sum is also a global function.
.set nomips16
.set nomicromips
.ent sum
.type sum, @function
sum:      # Label for the "sum" function.
.frame   $fp,8,$31      # vars= 0, regs= 1/0, args= 0, gp= 0
.mask 0x40000000,-4
.fmask 0x00000000,0
.set noreorder
.set nomacro
addiu $sp,$sp,-8        # Make space on stack for sum function.
sw    $fp,4($sp)        # Store old frame pointer on stack.
move  $fp,$sp           # Update frame pointer.
sw    $4,8($fp)
sw    $5,12($fp)
lw    $3,8($fp)
lw    $2,12($fp)
nop
addu  $2,$3,$2          # Actually do the operation of "a + b".
move  $sp,$fp
lw    $fp,4($sp)        # Restore the frame pointer
addiu $sp,$sp,8         # Restore the original stack pointer position.
j     $31               # Return from the sum function.
nop

.set macro
.set reorder
.end sum
.size sum, .-sum

```

```
.ident      "GCC: (OpenWrt GCC 5.3.0) 5.3.0"
```

It is not so easy to see how the original C code relates to the assembly language in the above listing.

A different way of obtaining this information is to compile the C code with debug information using "gcc -g test.c" to generate an a.out executable with all the symbol information intact (normally symbol information is "stipped out" of the final executable).

Then we can use `objdump` to **disassembly** (go from machine code to assembly language) the a.out executable file into assembly language with the C source code interspersed by using the command: "objdump -S a.out".

I have annotated the output below to put the original C code in red and also relevant sections of the assembly language in bold with some annotated comments. Note that in the disassembly below, register \$s8 is used as a synonym for \$fp (frame pointer). Also the disassembly generally refers to registers with their names without using a "\$".

You can see the resulting assembly language is more complex than the Harris & Harris book suggests but overall it has the same key structure.

```
a.out:      file format elf32-tradlittlemips
```

Disassembly of section .init:

```
0040043c <_init>:
 40043c: 27bdf0e0    addiu sp,sp,-32
 400440: afbc0018    sw     gp,24(sp)
 400444: afbf001c    sw     ra,28(sp)
 400448: 04110001    bal    400450 <_init+0x14>
 40044c: 00000000    nop
 400450: 0c100194    jal    400650 <frame_dummy>
 400454: 00000000    nop
 400458: 04110001    bal    400460 <_init+0x24>
 40045c: 00000000    nop
 400460: 0c1001d8    jal    400760 <__do_global_ctors_aux>
 400464: 00000000    nop
 400468: 8fbc0018    lw     gp,24(sp)
 40046c: 8fbf001c    lw     ra,28(sp)
 400470: 03e00008    jr     ra
 400474: 27bd0020    addiu sp,sp,32
```

Disassembly of section .text:

```

00400480 <__start>:
  400480: 04110004    bal    400494 <__start+0x14>
  400484: 0000f021    move   s8,zero
  400488: 00400488    0x400488
  40048c: 004004c0    0x4004c0
  400490: 004001a0    0x4001a0
  400494: 8ffc0000    lw     gp,0(ra)
  400498: 03fce023    subu   gp,ra,gp
  ... etc.   # I've truncated this initialization code since it is
              # lengthy and used to set up various aspects of running
              # C code within an operating system process.

# Main code starts here ...

00400690 <main>:          # main label indicates start of "main()".

int f, g, y; // global variables

int main(void)
{
  400690: 27bdf0e0    addiu  sp,sp,-32 # Make space on stack for use by
                                # the main function.
  400694: afbf001c    sw     ra,28(sp) # Push $ra onto the stack.
  400698: afbe0018    sw     s8,24(sp) # Push $fp onto stack.
  40069c: 03a0f021    move   s8,sp     # Set new frame pointer.
  4006a0: 3c1c0042    lui    gp,0x42   # Sets up "global pointer" for
  4006a4: 279c8830    addiu  gp,gp,-30672 # storing global variables.
  4006a8: afbc0010    sw     gp,16(sp) # Stores $gp onto stack.

  f = 2;
  4006ac: 8f828018    lw     v0,-32744(gp) # Address of "f" is obtained.
  4006b0: 24030002    li     v1,2        # Value of "2" is set in $v1.
  4006b4: ac430000    sw     v1,0(v0)     # Value written to f address.
  g = 3;
  4006b8: 8f82801c    lw     v0,-32740(gp) # Address of "g" ... etc.
  4006bc: 24030003    li     v1,3
  4006c0: ac430000    sw     v1,0(v0)

  y = sum(f, g);
  4006c4: 8f828018    lw     v0,-32744(gp) # This section reads in the
  4006c8: 00000000    nop                                # values of "f" and "g" from
  4006cc: 8c430000    lw     v1,0(v0)       # global data (saved above).
  4006d0: 8f82801c    lw     v0,-32740(gp)
  4006d4: 00000000    nop
  4006d8: 8c420000    lw     v0,0(v0)
  4006dc: 00000000    nop
  4006e0: 00402821    move   a1,v0        # Moves f & g into attribute

```

```

4006e4: 00602021    move    a0,v1           # registers $a0 & $a1 before
4006e8: 0c1001ca    jal     400728 <sum>    # calling the sum function.
4006ec: 00000000    nop
4006f0: 8fdc0010    lw      gp,16(s8)       # Restore $gp from stack.
4006f4: 00401821    move    v1,v0           # Moves sum result into $v1.
4006f8: 8f828020    lw      v0,-32736(gp)   # Gets address of "y".
4006fc: 00000000    nop
400700: ac430000    sw      v1,0(v0)        # Stores sum result into
                                # "y" address.

    return y;
400704: 8f828020    lw      v0,-32736(gp)   # Gets address of "y".
400708: 00000000    nop
40070c: 8c420000    lw      v0,0(v0)        # Load "y" result into $v0.
}

400710: 03c0e821    move    sp,s8
400714: 8fbf001c    lw      ra,28(sp)       # Restores $ra from stack.
400718: 8fbe0018    lw      s8,24(sp)       # Restores $fp from stack.
40071c: 27bd0020    addiu   sp,sp,32        # Restores the stack pointer.
400720: 03e00008    jr      ra              # Returns from main function.
400724: 00000000    nop

00400728 <sum>:

int sum(int a, int b) {
    400728: 27bdf8ff    addiu   sp,sp,-8        # Make space on stack for "sum".
    40072c: afbe0004    sw      s8,4(sp)        # Push stack frame register.
    400730: 03a0f021    move    s8,sp           # Set new stack frame pointer.
    400734: afc40008    sw      a0,8(s8)        # Push $a0 ("a") onto stack.
    400738: afc5000c    sw      a1,12(s8)       # Push $a1 ("b") onto stack.
    return (a + b);
    40073c: 8fc30008    lw      v1,8(s8)        # Load "a" and "b" values
    400740: 8fc2000c    lw      v0,12(s8)       # from stack.
    400744: 00000000    nop
    400748: 00621021    addu    v0,v1,v0        # Do "a + b" operation.
}

40074c: 03c0e821    move    sp,s8
400750: 8fbe0004    lw      s8,4(sp)        # Restore frame pointer.
400754: 27bd0008    addiu   sp,sp,8         # Restore stack pointer.
400758: 03e00008    jr      ra              # Jump back to return address.
40075c: 00000000    nop

00400760 <__do_global_ctors_aux>:
400760: 27bdf8d8    addiu   sp,sp,-40
400764: afb0001c    sw      s0,28(sp)
400768: 3c100041    lui     s0,0x41
40076c: afb10020    sw      s1,32(sp)
400770: afbf0024    sw      ra,36(sp)
400774: 26100810    addiu   s0,s0,2064

```

```

400778: 2411ffff    li    s1,-1
40077c: 8e190000    lw    t9,0(s0)
400780: 00000000    nop
400784: 13310005    beq   t9,s1,40079c <__do_global_ctors_aux+0x3c>
400788: 00000000    nop
40078c: 0320f809    jalr  t9
400790: 2610ffff    addiu s0,s0,-4
400794: 1000fff9    b     40077c <__do_global_ctors_aux+0x1c>
400798: 00000000    nop
40079c: 8fbf0024    lw    ra,36(sp)
4007a0: 8fb10020    lw    s1,32(sp)
4007a4: 8fb0001c    lw    s0,28(sp)
4007a8: 03e00008    jr    ra
4007ac: 27bd0028    addiu sp,sp,40

```

Disassembly of section .fini:

```

004007b0 <_fini>:
4007b0: 27bdf0e0    addiu sp,sp,-32
4007b4: afbc0018    sw    gp,24(sp)
4007b8: afbf001c    sw    ra,28(sp)
4007bc: 04110001    bal   4007c4 <_fini+0x14>
4007c0: 00000000    nop
4007c4: 0c100161    jal   400584 <__do_global_dtors_aux>
4007c8: 00000000    nop
4007cc: 8fbc0018    lw    gp,24(sp)
4007d0: 8fbf001c    lw    ra,28(sp)
4007d4: 03e00008    jr    ra
4007d8: 27bd0020    addiu sp,sp,32

```

Disassembly of section .plt:

```

004007e0 <_PROCEDURE_LINKAGE_TABLE>:
4007e0: 3c1c0041    lui   gp,0x41
4007e4: 8f990820    lw    t9,2080(gp)
4007e8: 279c0820    addiu gp,gp,2080
4007ec: 031cc023    subu  t8,t8,gp
4007f0: 03e07821    move  t7,ra
4007f4: 0018c082    srl   t8,t8,0x2
4007f8: 0320f809    jalr  t9
4007fc: 2718fffe    addiu t8,t8,-2

00400800 <__libc_start_main@plt>:
400800: 3c0f0041    lui   t7,0x41
400804: 8df90828    lw    t9,2088(t7)
400808: 03200008    jr    t9
40080c: 25f80828    addiu t8,t7,2088

```